

Selenium WebDriver

Complete Deep-Dive Notes

Topics: Dropdowns | Web Tables | Frames | Waits | JS Executor
With Real Demo Site URLs & Production Code Examples

Author: Prabhakar | Framework: OrangeHRM Selenium Framework
Generated: May 26, 2026

Table of Contents

1. Handling Dropdowns (Select, Custom div, Salesforce Lightning, Auto-suggest)
2. Handling Web Tables & Dynamic Tables (Pagination, Search, Infinite Scroll)
3. Handling Frames / iFrames (Static, Nested, Dynamic)
4. Waits in Selenium (Implicit, Explicit, Fluent, Custom)
5. JavaScriptExecutor (Click, Scroll, Set Value, Shadow DOM)

1. HANDLING DROPDOWNS IN SELENIUM

Most modern UIs (Salesforce Lightning, ServiceNow, Angular/React Material) do NOT use the HTML `<select>` tag. They render dropdowns as `<div>`, `<ul><li>` or `<input role='combobox'>`. The classic Select class will NOT work on them.

Demo Sites for Practicing Dropdowns

- * <https://the-internet.herokuapp.com/dropdown> (Native `<select>`)
- * <https://demoqa.com/select-menu> (React-select, multi-select)
- * <https://demoqa.com/auto-complete> (Auto-suggest typeahead)
- * <https://practice.expandtesting.com/dropdown> (Standard practice)
- * <https://ultimateqa.com/filling-out-forms/> (Cascading dropdowns)
- * <https://opensource-demo.orangehrmlive.com/> (Custom div, Login: Admin/admin123)

1.1 Native `<select>` - The Easy Case

Use Select class from `org.openqa.selenium.support.ui.Select`. Works ONLY when tag is literally `<select>`.

Example: <https://the-internet.herokuapp.com/dropdown>

```
driver.get("https://the-internet.herokuapp.com/dropdown");

WebElement dropdownEl = driver.findElement(By.id("dropdown"));
Select dropdown = new Select(dropdownEl);

// 3 ways to select:
dropdown.selectByVisibleText("Option 1");
dropdown.selectByValue("2");
dropdown.selectByIndex(1);

// Useful APIs:
dropdown.isMultiple();
dropdown.getFirstSelectedOption().getText();
dropdown.getOptions(); // all options
```

1.2 Custom `div/ul/li` Dropdowns (Real-World)

Recipe is ALWAYS 2 steps: (1) Click to open (2) Wait for options & select.

Example: <https://demoqa.com/select-menu>

```
driver.get("https://demoqa.com/select-menu");
// Click the react-select container to open
driver.findElement(By.id("withOptGroup")).click();
// Wait for option and click
By option = By.xpath("//div[contains(@class,'option') and text()='Group 1, option 1']");
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(option)).click();
```

Generic Reusable Method:

```

public void selectFromCustomDropdown(By toggle, By optionsBy, String value) {
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));
    wait.until(ExpectedConditions.elementToBeClickable(toggle)).click();
    List<WebElement> options = wait.until(
        ExpectedConditions.visibilityOfAllElementsLocatedBy(optionsBy));
    for (WebElement opt : options) {
        if (opt.getText().trim().equalsIgnoreCase(value)) {
            opt.click(); return;
        }
    }
    throw new NoSuchElementException("Option not found: " + value);
}

```

1.3 Salesforce Lightning Combobox

Lightning uses <input role='combobox'> + <div role='listbox'>. The input is READ-ONLY (must click).
Wait for aria-expanded='true' before selecting option.

```

public void selectLightningCombobox(String label, String optionText) {
    By combo = By.xpath("//lightning-combobox[.//label[normalize-space()='\"
        + label + \"']//input[@role='combobox']");
    WebElement el = wait.until(ExpectedConditions.elementToBeClickable(combo));
    el.click();
    wait.until(ExpectedConditions.attributeToBe(el, "aria-expanded", "true"));
    By opt = By.xpath("//lightning-base-combobox-item[.//span[normalize-space()='\"
        + optionText + \"']]");
    wait.until(ExpectedConditions.elementToBeClickable(opt)).click();
}

```

1.4 Auto-suggest / Typeahead

You must TYPE first, then AJAX returns suggestions dynamically.

Example: <https://demoqa.com/auto-complete>

```

driver.get("https://demoqa.com/auto-complete");
WebElement input = driver.findElement(By.id("autoCompleteMultipleInput"));
input.sendKeys("re");
By sugg = By.xpath("//div[contains(@class, 'option') and contains(text(), 'Red')]");
wait.until(ExpectedConditions.elementToBeClickable(sugg)).click();

```

1.5 Common Problems & Solutions

- * ElementNotInteractableException -> Wait for elementToBeClickable + scroll
- * StaleElementReferenceException -> Re-locate using By, never cache WebElement
- * Option not found (whitespace) -> Use normalize-space() in XPath
- * UnexpectedTagNameException -> It's NOT <select>; use custom approach
- * Dropdown closes before click -> Re-open inside loop for each pick

2. HANDLING WEB TABLES & DYNAMIC TABLES

Real challenge: pagination, sorting, filtering, lazy loading. Not the static table itself.

Demo Sites for Practicing Tables

- * <https://the-internet.herokuapp.com/tables> (Sortable + edit/delete)
- * <https://demoqa.com/webtables> (Dynamic add/edit/delete + pagination)
- * https://datatables.net/examples/basic_init/zero_configuration.html (jQuery DataTables: search, sort, pagination)
- * <https://demo.guru99.com/test/web-table-element.php> (Nested tables)
- * <https://www.ag-grid.com/example/> (AG-Grid virtualized enterprise)

2.1 Read All Data from Static Table

Example: <https://the-internet.herokuapp.com/tables>

```
driver.get("https://the-internet.herokuapp.com/tables");
List<WebElement> rows = driver.findElements(By.cssSelector("#table1 tbody tr"));
List<WebElement> headers = driver.findElements(By.cssSelector("#table1 thead th"));
System.out.println("Rows: " + rows.size() + ", Cols: " + headers.size());

// Read specific cell (row 2, col 3)
String cell = driver.findElement(
    By.cssSelector("#table1 tbody tr:nth-child(2) td:nth-child(3)")).getText();
```

2.2 Find Row by Value & Click Action (Classic Interview Q)

Question: Find the row where Last Name='Bach' and click Edit.

```
// Direct XPath approach:
By editLink = By.xpath(
    "//table[@id='table1']//tbody//tr[td[1][normalize-space()='Bach']]//a[text()='edit']");
driver.findElement(editLink).click();

// Production approach: dynamic column index from header name
// (See full method in HTML version)
```

2.3 Dynamic Tables with PAGINATION (Page 2/3 Problem)

Real scenario: The data is NOT on page 1. It appears only when you click page 2 or 3.

Example: https://datatables.net/examples/basic_init/zero_configuration.html

```

public boolean findAcrossPages(String searchName) {
    WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    while (true) {
        List<WebElement> rows = driver.findElements(By.cssSelector("#example tbody tr"));
        for (WebElement row : rows) {
            if (row.findElements(By.tagName("td")).get(0).getText().equals(searchName))
                return true; // Found!
        }
        WebElement next = driver.findElement(By.id("example_next"));
        if (next.getAttribute("class").contains("disabled")) return false;
        String firstRow = rows.get(0).getText();
        next.click();
        // CRITICAL: Wait for page to actually change!
        wait.until(ExpectedConditions.not(
            ExpectedConditions.textToBe(
                By.cssSelector("#example tbody tr:first-child"), firstRow)));
    }
}

```

KEY INSIGHTS for Pagination:

1. NEVER cache WebElement across pages (DOM rebuilt = StaleElement)
2. ALWAYS wait for 'first row text changed' after clicking Next
3. Check if Next button is disabled to stop the loop

2.4 Using Table Search (Best Practice)

```

// ALWAYS prefer search over walking pages when available!
driver.findElement(By.cssSelector("input[type='search']")).sendKeys("Tiger Nixon");
wait.until(d -> d.findElements(By.cssSelector("#example tbody tr")).size() == 1);

```

2.5 AG-Grid / Virtualized Tables

AG-Grid only renders VISIBLE rows. Rows below fold don't exist in DOM. Scroll the grid body via JS to load them.

```

// Scroll AG-Grid body:
js.executeScript("document.querySelector('.ag-body-viewport').scrollTop += 400;");
// Then re-query rows (new ones now exist in DOM)

```

2.6 Common Problems

- * StaleElement after Next -> Re-locate inside loop
- * XPath td[3] breaks -> Compute index from header dynamically
- * AG-Grid row not found -> Scroll body via JS
- * Pagination never ends -> Compare current to previous page text
- * getText() returns "" -> Use getAttribute('value') for inputs

3. HANDLING FRAMES / IFRAMES IN SELENIUM

A frame/iframe is 'a page inside a page'. Selenium only sees the top-level document. You must explicitly switch context to interact with frame elements.

Demo Sites for Practicing Frames

- * <https://the-internet.herokuapp.com/iframe> (TinyMCE editor in iframe)
- * https://the-internet.herokuapp.com/nested_frames (Nested frames)
- * <https://demoqa.com/frames> (Multiple iframes)
- * <https://demoqa.com/nestedframes> (Parent/child frames)
- * <https://practice.expandtesting.com/iframe> (Simple iframe)

3.1 All Ways to Switch INTO a Frame

- a) By index: `driver.switchTo().frame(0);` // least preferred
- b) By name/id: `driver.switchTo().frame('paymentFrame');`
- c) By WebElement: `driver.switchTo().frame(element);` // most reliable
- d) Wait+Switch (RECOMMENDED):

```
new WebDriverWait(driver, Duration.ofSeconds(15))
    .until(ExpectedConditions.frameToBeAvailableAndSwitchTo(By.id("mce_0_ifr")));
```

3.2 Switching OUT of a Frame

```
driver.switchTo().parentFrame(); // 1 level up
driver.switchTo().defaultContent(); // all the way to top (ALWAYS USE THIS)
```

3.3 Complete Working Example - TinyMCE Editor

URL: <https://the-internet.herokuapp.com/iframe>

```
driver.get("https://the-internet.herokuapp.com/iframe");
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

// Switch to editor iframe
wait.until(ExpectedConditions.frameToBeAvailableAndSwitchTo(By.id("mce_0_ifr")));

// Type in the editor
WebElement body = driver.findElement(By.tagName("body"));
body.clear();
body.sendKeys("Hello from Selenium!");

// ALWAYS come back to main page
driver.switchTo().defaultContent();
```

3.4 Nested Frames Example

URL: https://the-internet.herokuapp.com/nested_frames

```
driver.get("https://the-internet.herokuapp.com/nested_frames");
driver.switchTo().frame("frame-top"); // first level
driver.switchTo().frame("frame-middle"); // second level
String text = driver.findElement(By.id("content")).getText(); // "MIDDLE"
driver.switchTo().defaultContent(); // back to root
driver.switchTo().frame("frame-bottom"); // switch to sibling
```

3.5 Golden Rules

- * Cannot jump from one frame to sibling directly - go to defaultContent first
- * ALWAYS call defaultContent() after frame work (use try/finally)
- * If NoSuchElementException but locator is correct -> probably inside an iframe!

3.6 Common Problems

- * NoSuchElementException -> Frame not in DOM yet; use frameToBeAvailableAndSwitchTo
- * Can't click main page after frame -> Forgot defaultContent()
- * Iframe has no id/name -> Locate by src, title, pass WebElement
- * TinyMCE not accepting text -> switchTo frame first, sendKeys to body

4. WAITS IN SELENIUM

Most flaky tests are caused by a missing or wrong wait. Selenium runs at JVM speed while web apps load asynchronously (AJAX, animations, SPA routing).

Demo Sites for Practicing Waits

- * https://the-internet.herokuapp.com/dynamic_loading/1 (Hidden element)
- * https://the-internet.herokuapp.com/dynamic_loading/2 (Not yet rendered)
- * https://the-internet.herokuapp.com/dynamic_controls (Appear/disappear)
- * <https://demoqa.com/dynamic-properties> (Button enabled after delay)
- * <https://practice.expandtesting.com/dynamic-loading> (Loading spinner)

4.1 Implicit Wait

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));  
// Applies to EVERY findElement. Only checks DOM presence.  
// Does NOT wait for visibility/clickability!
```

CONS: Only handles 'element not in DOM'. Selenium warns against mixing with explicit waits (timeouts compound).

4.2 Explicit Wait - THE WORKHORSE

Example: https://the-internet.herokuapp.com/dynamic_loading/1

```
driver.get("https://the-internet.herokuapp.com/dynamic_loading/1");  
driver.findElement(By.cssSelector("#start button")).click();  
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(15));  
  
// Wait for spinner to disappear  
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("loading")));  
  
// Wait for result to be visible  
WebElement result = wait.until(  
    ExpectedConditions.visibilityOfElementLocated(By.cssSelector("#finish h4")));  
System.out.println(result.getText()); // "Hello World!"
```

ExpectedConditions You Use 90% of the Time:

- * `presenceOfElementLocated(by)` - exists in DOM (might be hidden)
- * `visibilityOfElementLocated(by)` - exists AND visible
- * `elementToBeClickable(by)` - visible AND enabled (use before click)
- * `invisibilityOfElementLocated(by)` - spinner/modal disappears
- * `textToBe(by, 'Done')` - wait until text matches
- * `attributeToBe(el, 'aria-expanded', 'true')` - combobox open
- * `frameToBeAvailableAndSwitchTo(by)` - iframes
- * `numberOfElementsToBeMoreThan(by, 0)` - table rows loaded
- * `stalenessOf(element)` - old element gone (page transition)

4.3 Fluent Wait

```
Wait<WebDriver> fluent = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
    .pollingEvery(Duration.ofMillis(250))          // faster polling
    .ignoring(NoSuchElementException.class)
    .ignoring(StaleElementReferenceException.class) // key!
    .withMessage("Element never loaded");
WebElement el = fluent.until(d -> d.findElement(By.id("target")));
```

Use FluentWait when: shorter polling needed, must ignore StaleElement, need custom error message.

4.4 Dynamic Controls Example

URL: https://the-internet.herokuapp.com/dynamic_controls

```
driver.get("https://the-internet.herokuapp.com/dynamic_controls");
driver.findElement(By.cssSelector("#checkbox-example button")).click();
wait.until(ExpectedConditions.invisibilityOfElementLocated(By.id("checkbox")));
wait.until(ExpectedConditions.textToBe(By.id("message"), "It's gone!"));

driver.findElement(By.cssSelector("#input-example button")).click();
WebElement input = wait.until(
    ExpectedConditions.elementToBeClickable(By.cssSelector("#input-example input")));
input.sendKeys("Now I can type!");
```

4.5 THE TRAP: Implicit + Explicit Together

NEVER MIX! When explicit wait polls findElement, implicit wait ALSO triggers for EACH poll.

WebDriverWait(10s) + implicit(10s) = up to 100s timeout!

FIX: Set implicit wait to 0, rely fully on explicit waits.

4.6 Common Problems

- * ElementClickInterceptedException -> Wait for overlay to disappear first
- * StaleElement repeatedly -> FluentWait with .ignoring(StaleElement)
- * Test super slow on CI -> Remove implicit wait, use explicit only
- * presenceOf works but click fails -> Element display:none; use elementToBeClickable

5. JAVASCRIPT EXECUTOR IN SELENIUM

JavaScriptExecutor lets you run arbitrary JS inside the browser. The ultimate fallback when normal Selenium actions fail, and great for scrolling/highlighting.

Demo Sites for Practicing JS Executor

- * https://the-internet.herokuapp.com/infinite_scroll (Scroll via JS)
- * <https://the-internet.herokuapp.com/shadowdom> (Shadow DOM access)
- * <https://demoqa.com/buttons> (JS click when intercepted)
- * <https://demoqa.com/date-picker> (Set date value via JS)
- * https://the-internet.herokuapp.com/drag_and_drop (HTML5 DnD via JS)

5.1 Basic Setup

```
JavaScriptExecutor js = (JavaScriptExecutor) driver;  
js.executeScript("alert('Hello!');");  
  
// executeScript(script, args...) - synchronous  
// executeAsyncScript(script, args...) - async (must call callback)
```

5.2 JS Click (Bypass Overlays)

Example: <https://demoqa.com/buttons>

```
driver.get("https://demoqa.com/buttons");  
WebElement btn = driver.findElement(By.xpath("//button[text()='Click Me']"));  
// Normal click fails due to overlay:  
// btn.click(); // ElementClickInterceptedException!  
// JS click bypasses:  
((JavaScriptExecutor) driver).executeScript("arguments[0].click();", btn);
```

5.3 Scroll Into View

Example: https://the-internet.herokuapp.com/infinite_scroll

```
driver.get("https://the-internet.herokuapp.com/infinite_scroll");  
JavaScriptExecutor js = (JavaScriptExecutor) driver;  
// Scroll to bottom to trigger lazy-load:  
js.executeScript("window.scrollTo(0, document.body.scrollHeight);");  
  
// Scroll specific element into view:  
js.executeScript("arguments[0].scrollIntoView({block:'center'});", element);
```

5.4 Set Value on Date Pickers (sendKeys won't work)

Example: <https://demoqa.com/date-picker>

```

driver.get("https://demoqa.com/date-picker");
WebElement dateInput = driver.findElement(By.id("datePickerMonthYearInput"));
// JS set value + trigger React events:
js.executeScript(
    "arguments[0].value = arguments[1];"
    + "arguments[0].dispatchEvent(new Event('input', {bubbles:true}));"
    + "arguments[0].dispatchEvent(new Event('change', {bubbles:true}));",
    dateInput, "05/26/2026");

```

WHY dispatch events? React/Angular don't notice raw value= writes. Firing input/change events makes the framework re-render correctly.

5.5 Shadow DOM Access

Example: <https://the-internet.herokuapp.com/shadowdom>

```

driver.get("https://the-internet.herokuapp.com/shadowdom");
WebElement host = driver.findElement(By.cssSelector("my-paragraph:nth-child(1)"));

// Pre-Selenium 4: JS only way
WebElement inner = (WebElement) js.executeScript(
    "return arguments[0].shadowRoot.querySelector('slot')", host);

// Selenium 4 native:
SearchContext shadow = host.getShadowRoot();
WebElement slot = shadow.findElement(By.cssSelector("slot"));

```

5.6 Other Useful JS Recipes

- * Highlight: `js.executeScript('arguments[0].style.border="3px solid green"', el)`
- * Page title: (String) `js.executeScript('return document.title;')`
- * Page ready: `js.executeScript('return document.readyState === "complete";')`
- * Open tab: `js.executeScript('window.open(url, "_blank");')`
- * Zoom: `js.executeScript('document.body.style.zoom="80%";')`
- * Remove element: `js.executeScript('arguments[0].remove();', banner)`

5.7 Common Problems

- * `ElementClickIntercepted` (cookie banner) -> `Dismiss` OR `jsClick`
- * Date picker rejects `sendKeys` -> `jsSetValue` with `dispatchEvent`
- * Element below fold, fails headless -> `scrollIntoView` first
- * `executeAsyncScript` hangs -> Forgot to call callback argument
- * 'Cannot read properties of null' -> Element not in DOM yet; add wait

=== END OF NOTES ===

Generated for SeleniumFramework | Prabhakar | May 2026

For full HTML version with tables/formatting: see [Selenium-Deep-Dive-Notes.html](#)